



Name:	Lyka Sophia S. Jardeleza	Date:	05/03/2025
Program:	BSCpE	Section:	A361

A. Coding Exercise:

(Please include the following: **1.** Photos and videos of the physical test outputs, **2.** Photos of the codes that were inputted in the Quartus Prime software, and **3.** The raw source code that you made for the coding exercise.)

Coding Exercise: Implement a 16-bit BCD Decoder Using Shift-Add 3 Algorithm

1. Photos of Physical Test Outputs	2. Photos of Codes
Laboratory3 CodingExercise.mp4	<pre> 1 module shift_add_three (2 input [15:0] binary, 3 output reg [3:0] bcd0, 4 output reg [3:0] bcd1, 5 output reg [3:0] bcd2, 6 output reg [3:0] bcd3, 7 output reg [3:0] bcd4 8); 9 integer i; 10 reg [39:0] shift_reg; 11 12 always @ (binary) begin 13 shift_reg = 40'b0; 14 shift_reg[15:0] = binary; 15 16 for (i = 0; i < 16; i = i + 1) begin 17 if (shift_reg[19:16] >= 5) shift_reg[19:16] = shift_reg[19:16] + 3; 18 if (shift_reg[23:20] >= 5) shift_reg[23:20] = shift_reg[23:20] + 3; 19 if (shift_reg[27:24] >= 5) shift_reg[27:24] = shift_reg[27:24] + 3; 20 if (shift_reg[31:28] >= 5) shift_reg[31:28] = shift_reg[31:28] + 3; 21 if (shift_reg[35:32] >= 5) shift_reg[35:32] = shift_reg[35:32] + 3; 22 23 shift_reg = shift_reg << 1; 24 25 end 26 27 bcd4 = shift_reg[35:32]; 28 bcd3 = shift_reg[31:28]; 29 bcd2 = shift_reg[27:24]; 30 bcd1 = shift_reg[23:20]; 31 bcd0 = shift_reg[19:16]; 32 end 33 endmodule 34 35 module seven_segment (36 input [3:0] digit, 37 output reg [6:0] seg 38); 39 always @(*) begin 40 case (digit) 41 4'd0: seg = 7'b100_0000; 42 4'd1: seg = 7'b111_1001; 43 4'd2: seg = 7'b010_0100; 44 4'd3: seg = 7'b011_0000; 45 4'd4: seg = 7'b001_1001; 46 4'd5: seg = 7'b001_0010; 47 4'd6: seg = 7'b000_0010; 48 4'd7: seg = 7'b111_1000; 49 4'd8: seg = 7'b000_0000; 50 4'd9: seg = 7'b001_0000; 51 default: seg = 7'b111_1111; 52 endcase 53 end 54 endmodule 55 56 module seven_segment (57 input [3:0] digit, 58 output reg [6:0] seg 59); 60 always @(*) begin 61 case (digit) 62 4'd0: seg = 7'b100_0000; 63 4'd1: seg = 7'b111_1001; 64 4'd2: seg = 7'b010_0100; 65 4'd3: seg = 7'b011_0000; 66 4'd4: seg = 7'b001_1001; 67 4'd5: seg = 7'b001_0010; 68 4'd6: seg = 7'b000_0010; 69 4'd7: seg = 7'b111_1000; 70 4'd8: seg = 7'b000_0000; 71 4'd9: seg = 7'b001_0000; 72 default: seg = 7'b111_1111; 73 endcase 74 end 75 endmodule 76 77 module top_module (78 input [3:0] SW, 79 output [9:0] LEDR, 80 output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4 81); 82 reg [7:0] A = 0, B = 0; 83 reg [15:0] result; 84 wire [3:0] bcd0, bcd1, bcd2, bcd3, bcd4; 85 86 // LED output 87 assign LEDR[9:8] = SW[9:8]; 88 assign LEDR[7:0] = result[7:0]; 89 90 // FSM logic 91 always @(*) begin 92 case (SW[9:8]) 93 2'b00: result = {8'b0, SW[7:0]}; // Input A 94 2'b01: result = {8'b0, SW[7:0]}; // Input B 95 2'b11: result = A * B; // Multiply 96 default: result = 0; 97 endcase 98 end 99 100 // BCD converter 101 shift_add_three bcd_converter (102 .binary(result), 103 .bcd0(bcd0), 104 .bcd1(bcd1), 105 .bcd2(bcd2), 106 .bcd3(bcd3), 107 .bcd4(bcd4) 108); 109 110 // Display 111 seven_segment s0(bcd0, HEX0); 112 seven_segment s1(bcd1, HEX1); 113 seven_segment s2(bcd2, HEX2); 114 seven_segment s3(bcd3, HEX3); 115 seven_segment s4(bcd4, HEX4); 116 endmodule </pre>



Raw Source Code:

```
module shift_add_three (
    input [15:0] binary,
    output reg [3:0] bcd0,
    output reg [3:0] bcd1,
    output reg [3:0] bcd2,
    output reg [3:0] bcd3,
    output reg [3:0] bcd4
);
    integer i;
    reg [39:0] shift_reg;

    always @ (binary) begin
        shift_reg = 40'b0;
        shift_reg[15:0] = binary;

        for (i = 0; i < 16; i = i + 1) begin
            if (shift_reg[19:16] >= 5) shift_reg[19:16] = shift_reg[19:16] + 3;
            if (shift_reg[23:20] >= 5) shift_reg[23:20] = shift_reg[23:20] + 3;
            if (shift_reg[27:24] >= 5) shift_reg[27:24] = shift_reg[27:24] + 3;
            if (shift_reg[31:28] >= 5) shift_reg[31:28] = shift_reg[31:28] + 3;
            if (shift_reg[35:32] >= 5) shift_reg[35:32] = shift_reg[35:32] + 3;

            shift_reg = shift_reg << 1;
        end

        bcd4 = shift_reg[35:32];
        bcd3 = shift_reg[31:28];
        bcd2 = shift_reg[27:24];
        bcd1 = shift_reg[23:20];
        bcd0 = shift_reg[19:16];
    end
endmodule

module seven_segment (
    input [3:0] digit,
    output reg [6:0] seg
);
    always @(*) begin
        case (digit)
            4'd0: seg = 7'b100_0000;
            4'd1: seg = 7'b111_1001;
            4'd2: seg = 7'b010_0100;
            4'd3: seg = 7'b011_0000;
            4'd4: seg = 7'b001_1001;
            4'd5: seg = 7'b001_0010;
            4'd6: seg = 7'b000_0010;
            4'd7: seg = 7'b111_1000;
            4'd8: seg = 7'b000_0000;
            4'd9: seg = 7'b001_0000;
            default: seg = 7'b111_1111;
        endcase
    end
endmodule

module top_module (
    input [9:0] SW,
    output [9:0] LEDR,
    output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4
);
    reg [7:0] A = 0, B = 0;
    reg [15:0] result;
    wire [3:0] bcd0, bcd1, bcd2, bcd3, bcd4;

    // LED output
    assign LEDR[9:8] = SW[9:8];
```



```
assign LEDR[7:0] = result[7:0];

// FSM logic
always @(*) begin
    case (SW[9:8])
        2'b00: result = {8'b0, SW[7:0]}; // Input A
        2'b01: result = {8'b0, SW[7:0]}; // Input B
        2'b11: result = A * B;           // Multiply
        default: result = 0;
    endcase
end

always @(*) begin
    case (SW[9:8])
        2'b00: A = SW[7:0];
        2'b01: B = SW[7:0];
    endcase
end

// BCD converter
shift_add_three bcd_converter (
    .binary(result),
    .bcd0(bcd0),
    .bcd1(bcd1),
    .bcd2(bcd2),
    .bcd3(bcd3),
    .bcd4(bcd4)
);

// Display
seven_segment s0(bcd0, HEX0);
seven_segment s1(bcd1, HEX1);
seven_segment s2(bcd2, HEX2);
seven_segment s3(bcd3, HEX3);
seven_segment s4(bcd4, HEX4);
endmodule
```

B. Findings ,Observations, and Comments:

Guiding Questions :

1. Did your group solve the coding exercise without the help of other group/s?
2. Are there any difficulties during coding or set-up that you encountered?
3. What are your findings, observations, and comments in the coding exercise?

Throughout the coding exercise, I have resolved or figured out how it should really work with the assistance of my friends. There were difficulties in figuring out how the code or exercise should work and when I figured it out, I enjoyed trying different numbers to multiply. I have learned how to solve manually the Shift-Add 3 Algorithm and on how to apply it in the actual coding exercise. It was kind of difficult at first since it really looks complicated to apply in the code. I observed how multiplication of two 8-bit numbers using switches can be visualized on 7-segment displays by converting the binary result to BCD using the Shift-Add-3 Algorithm. I learned the importance of understanding binary manipulation, especially when shifting bits and checking conditions inside a loop.